POLITECNICO
MILANO 1863

ANTLAB

Internet of Things Lab

Lab 3: HTTP vs. CoAP

Contacts

Fabio Palmese

Mail: fabio.palmese@polimi.it

DEIB – ANTLab (Building 21 Floor 5)

Webex:
https://politecnicomilano.webex.com/meet/fabio.palmese

ANTLAB

Agenda

- HTTP basics
- CoAP basics
- CoAP in action
- Packet inspection with Wireshark

Why different protocols?

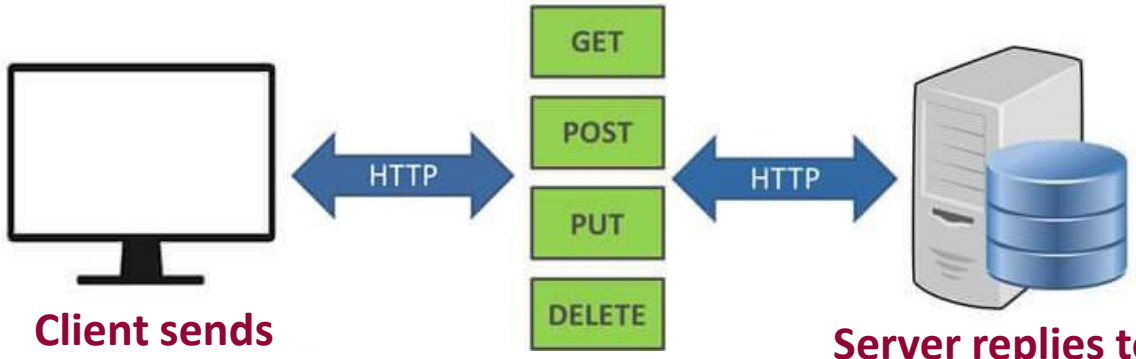
Different needs...

- End-user cloud applications
  - Display sensor data on mobile app, website dashboards
    - HTTP, Websockets…

- Local sensor networks
  - Transmit sensor data
    - (Usually) over **802.15.4** (zigbee), Bluetooth low energy **BLE**

HTTP methods in a nutshell

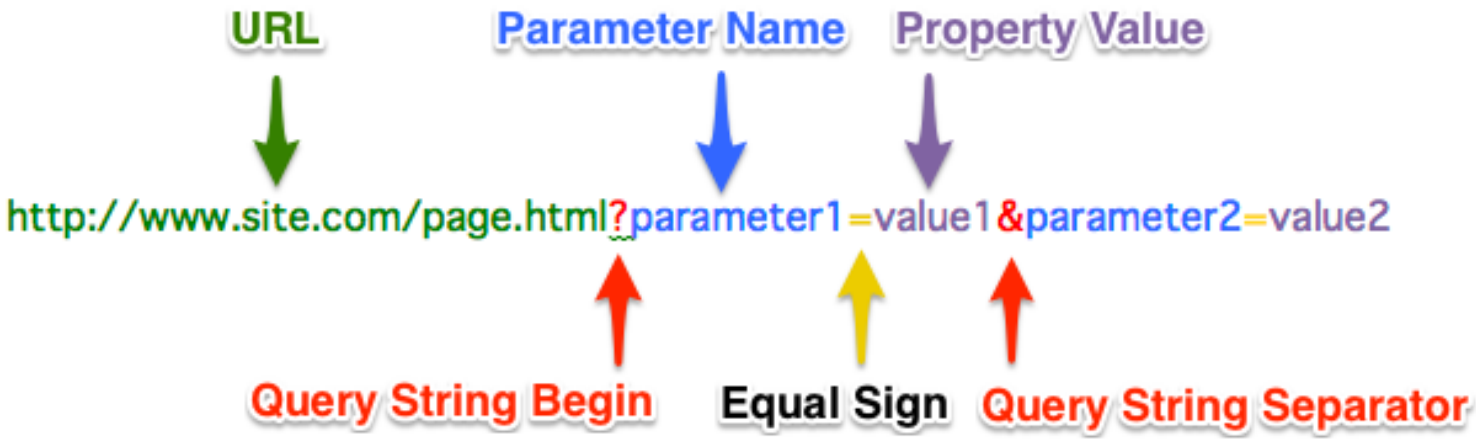
- Client-server achitecture
- Restful API methods
  - GET: read resource
  - POST: create new resource
  - PUT: update resource
  - DELETE: delete resource



ANTLAB

Uniform Resource Identifiers (URI)

- https://www.example.com/
- http://www.example.com:1234/
- https://www.example.com:1234/forum/question
- https://www.example.com:1234/forum/question?tag=networking



ANTLAB

How to send HTTP requests?

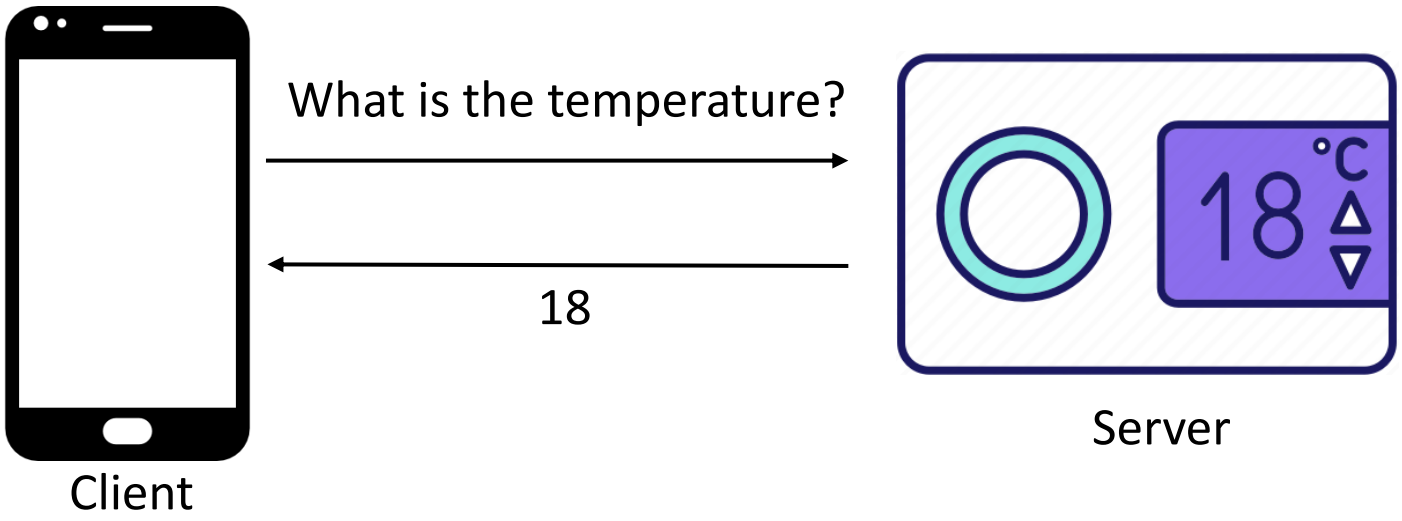
Several ways to send an HTTP request to a server:

- Via browser (https://api.thecatapi.com/v1/images/search)
- GUI clients: Postman (https://docs.api.getpostman.com)
- Code library (C, python, C++, Java, JavaScript…)
- **Command line** *(with **curl**)*

cURL

- Runs in the terminal
- Simple **GET** request: ***curl*** http://hostname:port/path
- -X specify custom method

  curl –X GET http://0.0.0.0:5000/living_room/temperature

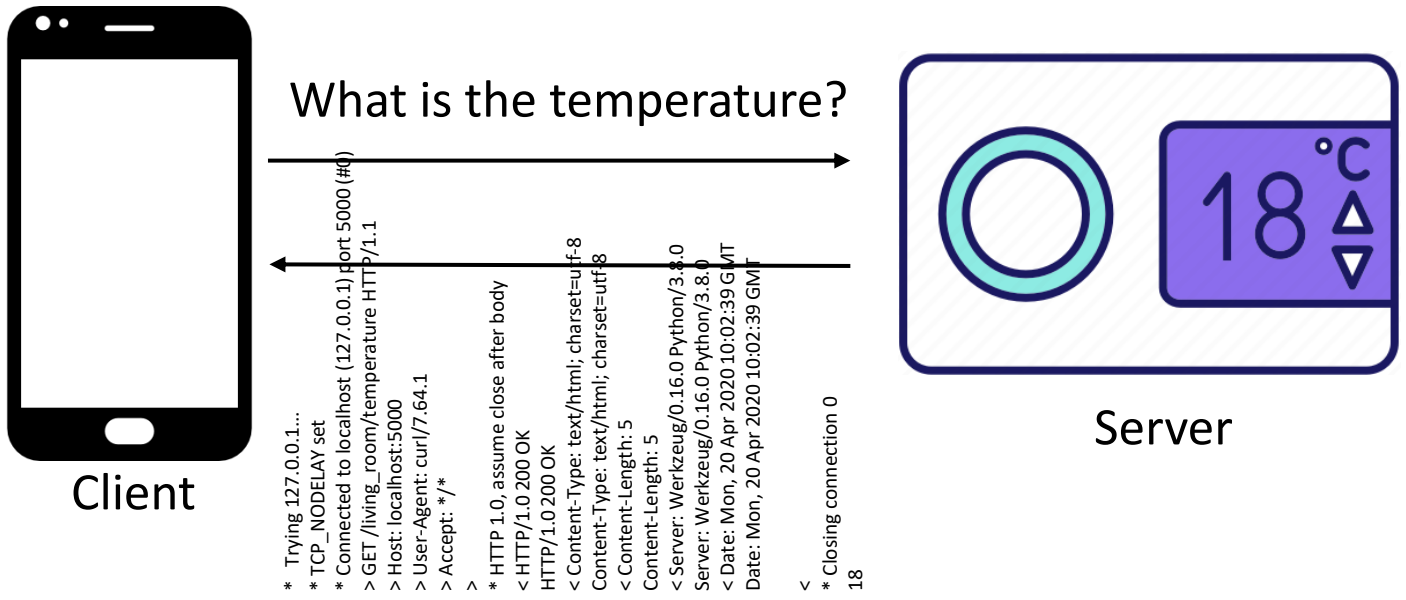


https://github.com/fpalmese/IoT2023/tree/main/HTTPServer

ANTLAB

What's really happening?

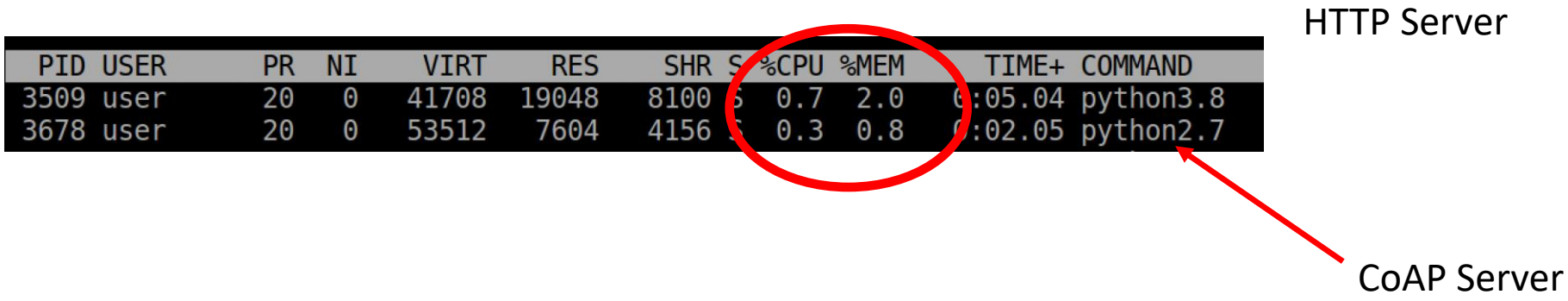
- Request header + Response header
- -v option in *curl* include the headers
- Roughly 370 bytes for the request/response
- **Too verbose!**



ANTLAB

HTTP Problems

- Long messages
- Resource-hungry



Drawbacks in IoT

Target devices:

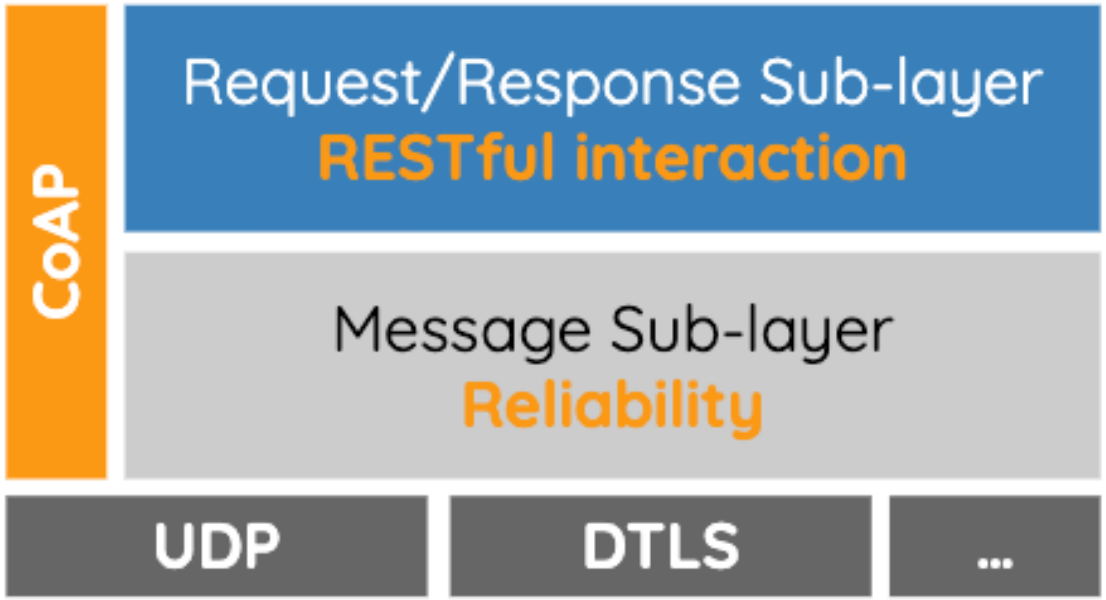
- Battery-powered
- ≈ 1$
- ≈ 100 KB Flash
- ≈ 100 KB RAM

HTTP is not a good fit!

ANTLAB

COnstrained Application Protocol (CoAP)

- Fits to the small resources of IoT devices
- RESTful calls
- Only needs small memories and slow processor
- Uses the UDP protocol
- M2M support



ANTLAB

TCP vs. UDP

TCP

Reliable transport

UDP

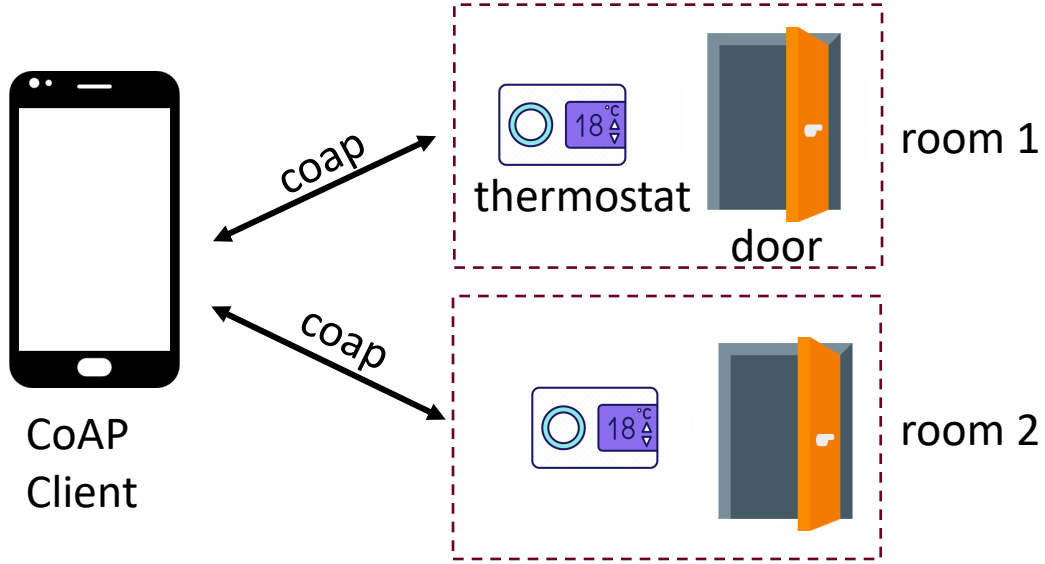
Unreliable transport

CoAP Features

- RESTful API
- UDP datagram based
- Resource Discovery mode
- Observe Resources
- CONfirmable/NON-Confirmable messages (CON/NON)
- Block transfer mode

CoAP in Action

- Smart home scenario (based on https://github.com/fpalmese/IoT2023/tree/main/CoAPServer)
- Discovery
- Read the temperature once - > GET
- Read every change of temperature -> Observe
- Close the living room door -> PUT
- Add a smart door -> POST



Accessing CoAP Resources – COAP-CLI

- Open the command line
- Let's do our first request:

coap METHOD URI *(coap –h for list of features)*

(local server)

*coap get coap://**localhost:5683**/living_room/temperature*

(public server)

*coap post coap://**coap.me**/whatever*

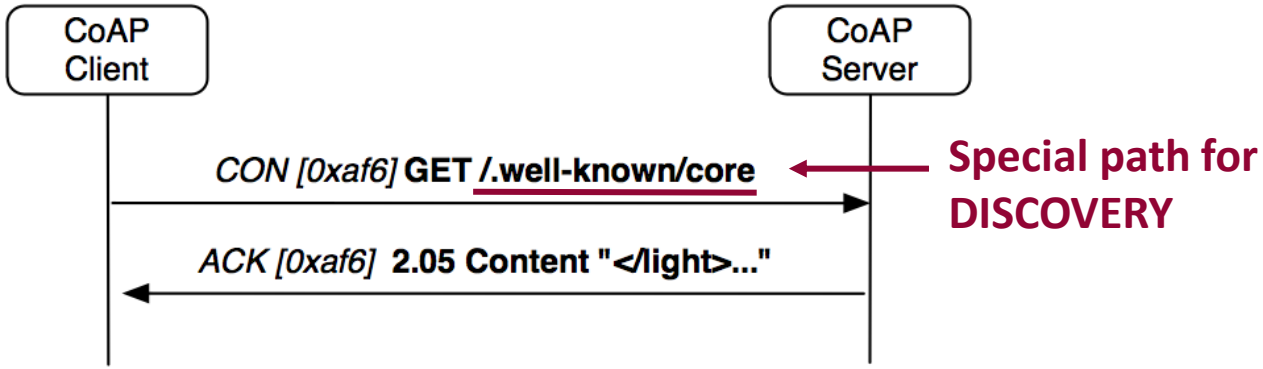
CoAP Requests/Operations

- DISCOVERY: show available resources
- GET: read the value of a resource
- POST/PUT: change the value of a resource
- POST: create a new resource
- DELETE: delete a resource
- OBSERVE: obtain all new values of a resource

CoRE Resource Discovery

A special GET request which asks for the available resources

GET request directed to the resource **/.well-known/core**




```
user@user-iot:~$ coap get coap://localhost:5683/.well-known/core
(2.05)  </basic>;ct=0;if="if1";rt="rt1",</dining_room>;ct=0;if="if1";rt="rt1",</
dining_room/door>;ct=0,</dining_room/temperature>;obs,</hello_post>;ct=0,</hello
_world>;ct=0,</living_room>;ct=0;if="if1";rt="rt1",</living_room/door>;ct=0,</li
ving_room/temperature>;obs,</main_door>;ct=0,</test>;obs,
```

Some Examples

- Read a resource value with GET:

  coap get coap://localhost:5683/dining_room/temperature

- Update a resource value with PUT (with URI query):

  coap put coap://localhost:5684/living_room/door?status=CLOSED

- Update a resource value with POST (with payload):

  coap post coap://localhost:5683/hello_post –p "hello"

- Create a resource with POST (with query and payload):

  coap post coap://localhost:5683/living_room?create -p "new_door"

- Delete a resource with DELETE

  coap delete coap://localhost:5683/living_room/door

Observe Request

Special GET with option ***observe***: receive continuously updates
Can only be done to OBSERVABLE resources

coap get coap://localhost:5683/living_room/temperature -o

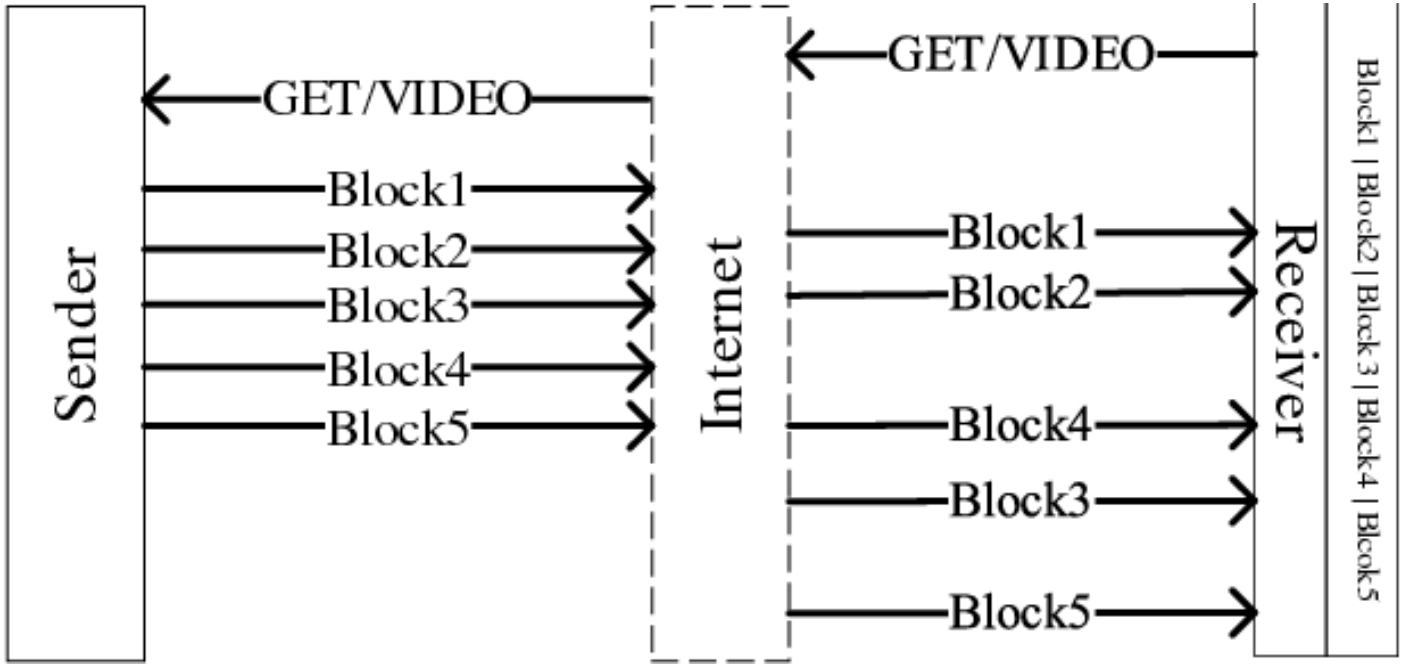
```
user@user-iot:~$ coap get coap://localhost:5683/living_room/temperature -o
(2.05)  19.82
(2.05)  31.79
(2.05)  22.3
(2.05)  21.01
```

We keep receiving new values with only one request!

Block Transfer Mode

Send a response into separate application messages

- If the resource is big, transfer it using several application messages



- If a block is lost? Retransmit **only the single block** (if request is **CON**)

CoAP Response Codes

	Code	Description	Reference
SUCCESS (2.xx)	2.01	Created	[RFC7252]
	2.02	Deleted	[RFC7252]
	2.03	Valid	[RFC7252]
	2.04	Changed	[RFC7252]
	2.05	Content	[RFC7252]
CLIENT ERROR (4.xx)	4.00	Bad Request	[RFC7252]
	4.01	Unauthorized	[RFC7252]
	4.02	Bad Option	[RFC7252]
	4.03	Forbidden	[RFC7252]
	4.04	Not Found	[RFC7252]
	4.05	Method Not Allowed	[RFC7252]
	4.06	Not Acceptable	[RFC7252]
	4.12	Precondition Failed	[RFC7252]
	4.13	Request Entity Too Large	[RFC7252]
	4.15	Unsupported Content-Format	[RFC7252]
SERVER ERROR (5.xx)	5.00	Internal Server Error	[RFC7252]
	5.01	Not Implemented	[RFC7252]
	5.02	Bad Gateway	[RFC7252]
	5.03	Service Unavailable	[RFC7252]
	5.04	Gateway Timeout	[RFC7252]
	5.05	Proxying Not Supported	[RFC7252]

ANTLAB

CoAP vs. HTTP

COAP	HTTP
UDP	TCP
RESTful API	RESTful API
Constrained devices	Servers/PC
Small packets	Big packets
Small overhead	Huge overhead
Resource discovery	x
Observe pattern	x
CON/NON-CON Acks	Handled by TCP
5683/5684	80/443
Still in progress/young protocol	Mature protocol

POLITECNICO MILANO 1863

ANTLAB

Wireshark

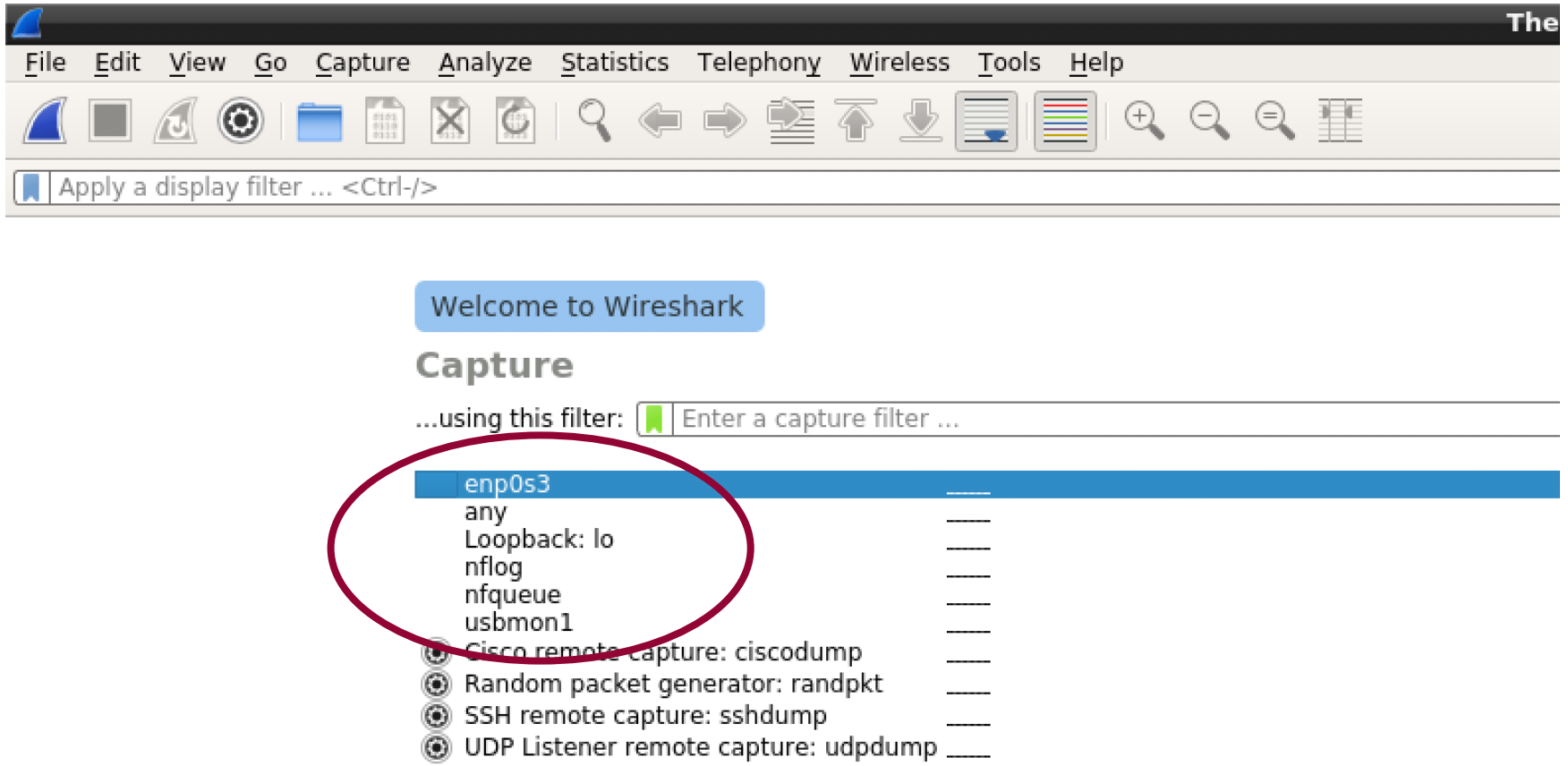
CoAP Packet Inspection

Wireshark

- Open source packet analyzer
  - Traffic analysis
  - Packet analysis
  - Protocol analysis
  - Sniffing

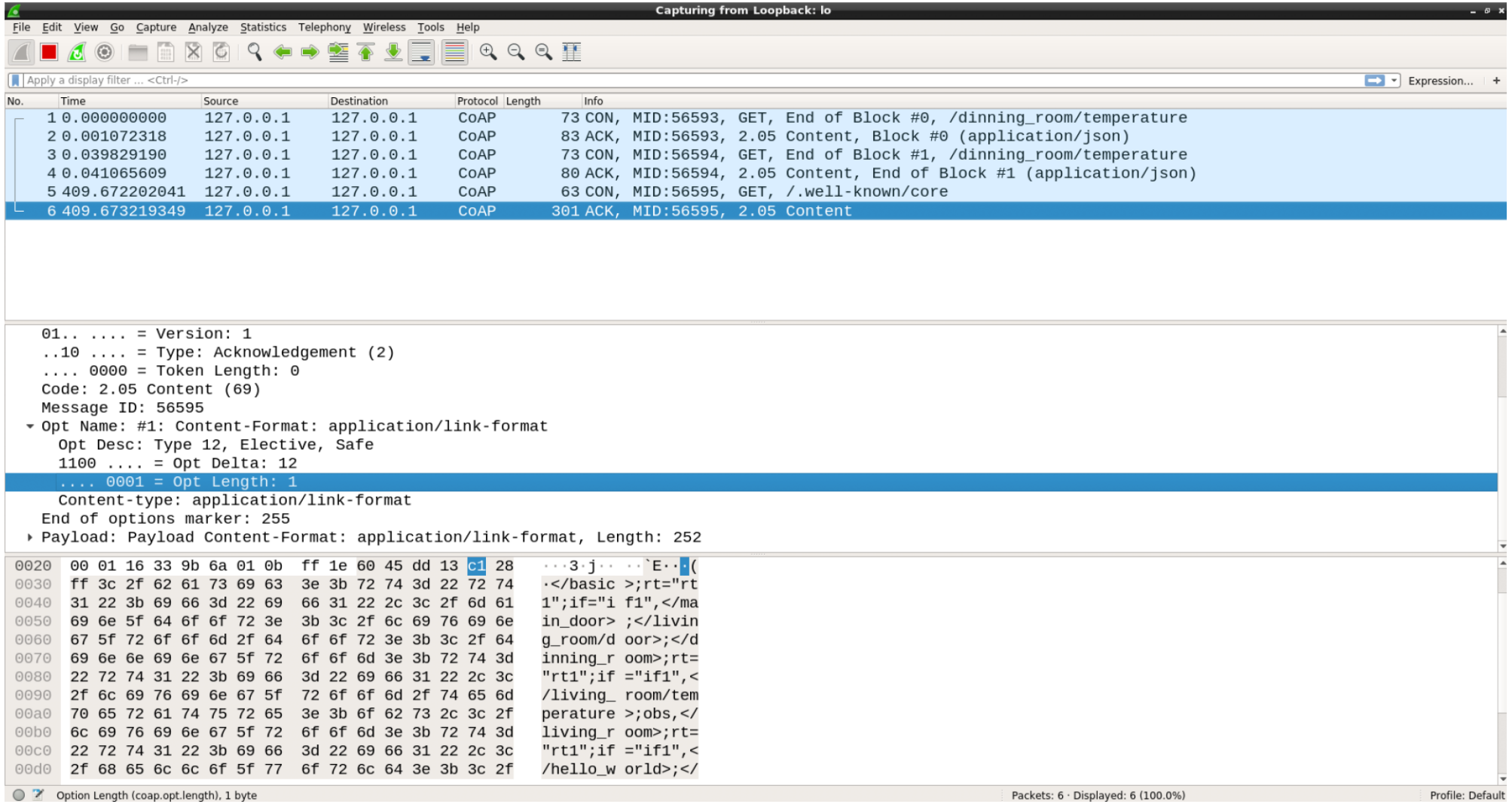
- Process of capturing, decoding and analyzing network traffic
  - What is the network traffic pattern
  - How traffic flows among nodes

Wireshark



ANTLAB

Packet Dissector



ANTLAB

Some Filters…

- ip.src -> source IP address
- ip.dst -> destination IP address
- ip.addr -> IP address (source or destination)
- eth.dst -> Destination MAC address
- tcp.port -> source or destination port
- udp, HTTP, coap, mqtt, …
- …

Some Operators…

- Equal: == or eq
- Not Equal: != or ne
- Greater than: > or gt
- Less than: < or lt
- Greater than or equal: >= ge
- Less than or equal: <= le
- Binary operators (and &&, or ||)
- …

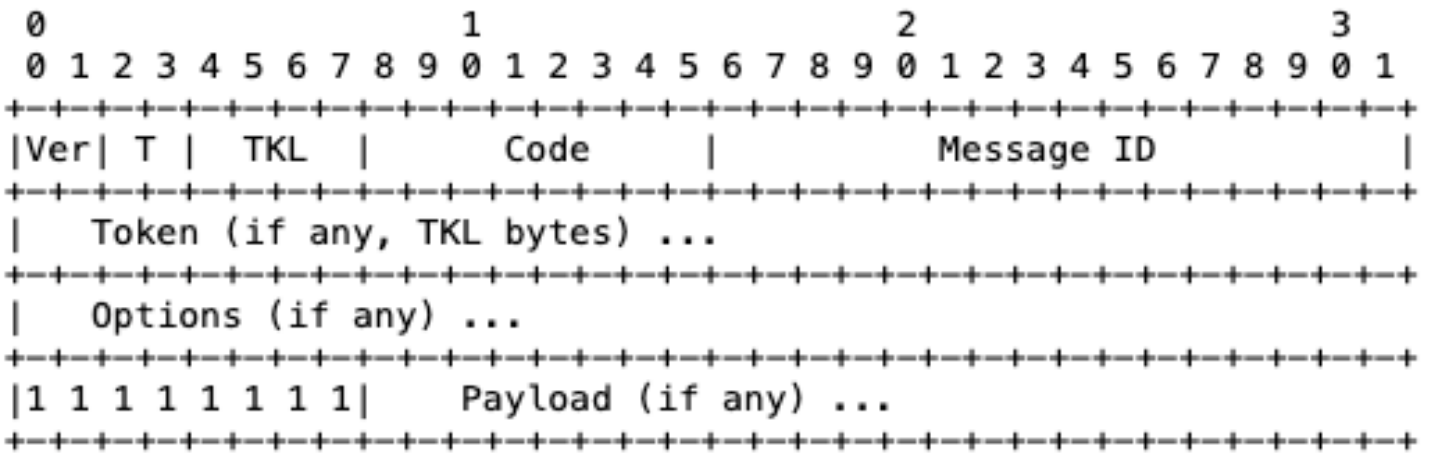
Examples

- ip.addr == 192.168.3.90 — only packets involving device with IP 192.168.3.90
- ip.src == 10.79.0.123 — only packets with source device with IP 10.79.0.123
- http || coap — only HTTP or COAP packets
- tcp.port == 80 && http.request — only HTTP requests on port 80

Please Note: If you don't see HTTP packets in Wireshark do:
Edit -> Preferences -> Protocols -> HTTP and add the port you are using for HTTP

ANTLAB

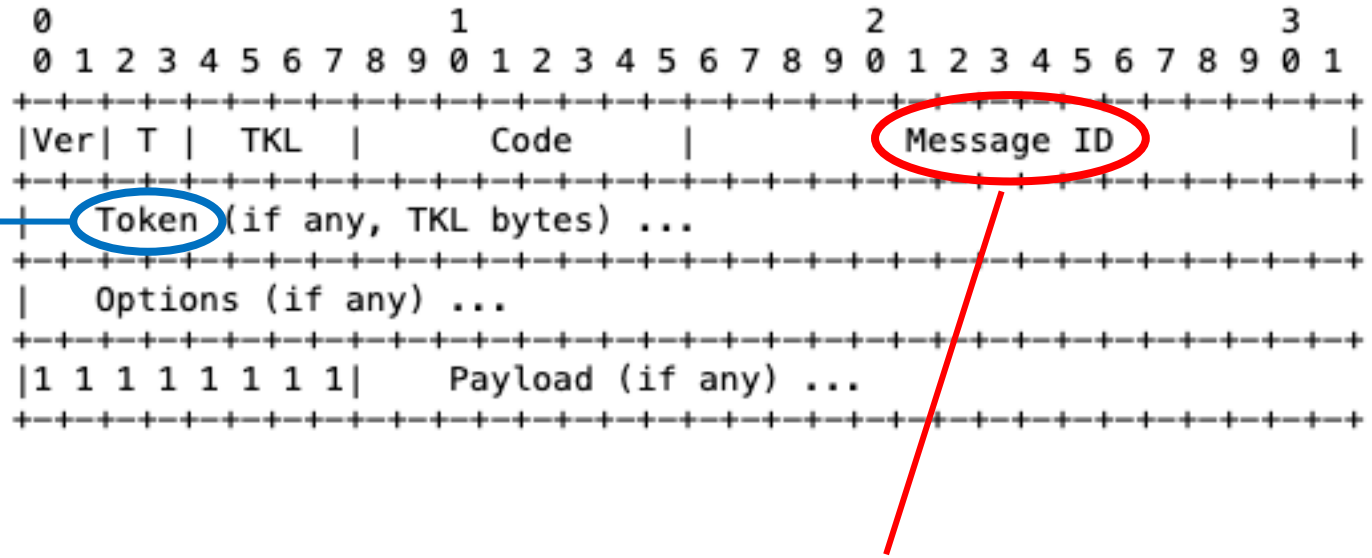
CoAP Message Format



- Ver: CoAP version number
- T - Type:
  - 0) confirmable (CON)
  - 1) Non-confirmable (NON)
  - 2) Acknowledgement (ACK)
  - 3) Reset (RST)
- TKL: token length
- Code (Response Code or Request Method)
- Message ID (MID): sequential number (next slide)

ANTLAB

CoAP Message Format

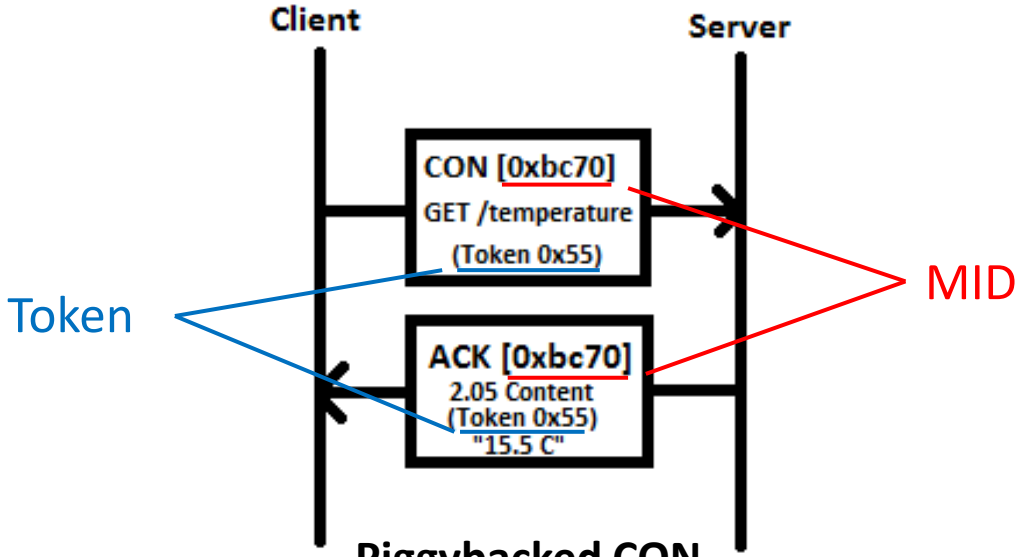


Token

Associates a **Request** with the corresponding **Response(s)**

Message ID (MID)

Associates a **CON** message to the corresponding **ACK** message



Piggybacked CON
(Same MID, Same Token)

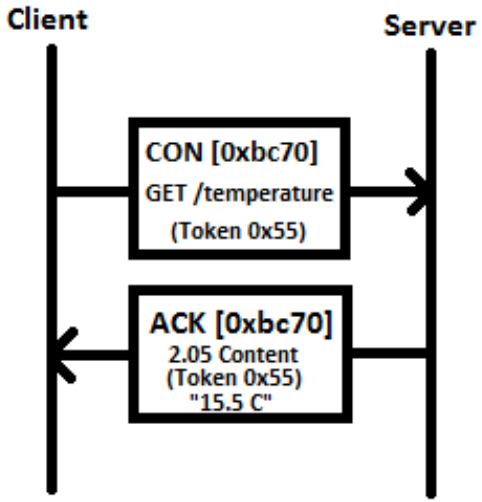
Piggybacked NON:
Different MID, Same Token

Separate CON/NON:
Different MID, Same Token

ANTLAB

Piggybacked vs. Separate Response

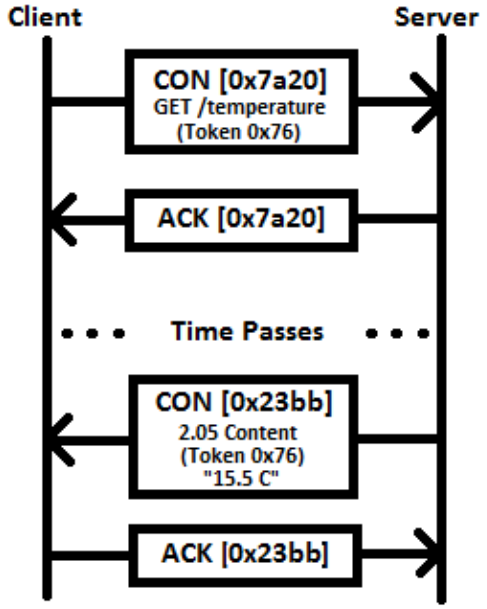
Piggybacked Response



Response is directly in the ACK

SAME MID AND TOKEN

Separate Response



Response is sent in a successive moment with a new message (SAME TOKEN, DIFFERENT MID)

ANTLAB

Some CoAP Filters

- **coap.code == *X***
  - ***X*** can be a **request method** code
  - ***X*** can be a **response code**
- **coap.type == *Y***
  - ***Y*** represents **message type**
    - 0 = CON
    - 1 = NON
    - 2 = ACK
    - 3 = RST

Code	Name
1	GET
2	POST
3	PUT
4	DELETE

CoAP Request codes

Code	Description
65	2.01 Created
66	2.02 Deleted
67	2.03 Valid
68	2.04 Changed
69	2.05 Content
128	4.00 Bad Request
129	4.01 Unauthorized
130	4.02 Bad Option
131	4.03 Forbidden
132	4.04 Not Found
133	4.05 Method Not Allowed
134	4.06 Not Acceptable
140	4.12 Precondition Failed
141	4.13 Request Entity Too Large
143	4.15 Unsupported Media Type
160	5.00 Internal Server Error
161	5.01 Not Implemented
162	5.02 Bad Gateway
163	5.03 Service Unavailable
164	5.04 Gateway Timeout
165	5.05 Proxying Not Supported

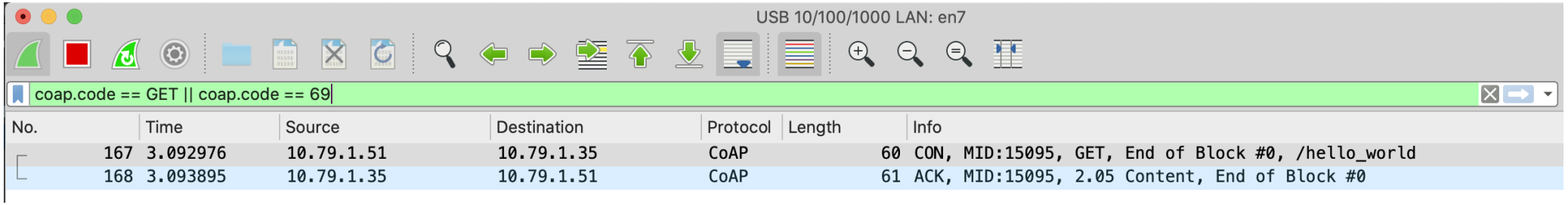
CoAP Response Codes

More CoAP Filters…

- coap.mid
- coap.code
- coap.length
- coap.opt.observe
- coap.opt.uri_path
- coap.opt.uri_port
- coap.opt.uri_query
- coap.payload
- coap.payload_length
- For a more detailed reference: https://www.wireshark.org/docs/dfref/c/coap.html

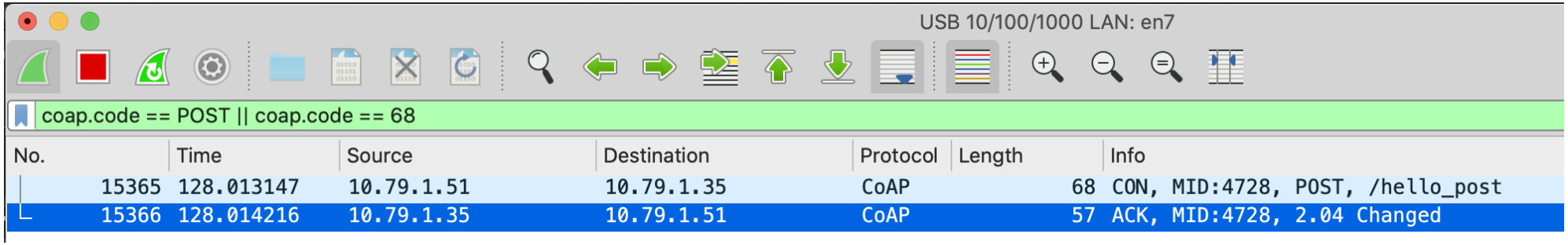
Filter: CoAP Methods

- Insert the filter: coap.code == GET || coap.code == 69



We obtain all packets containing CoAP GET Requests or 2.05 Responses

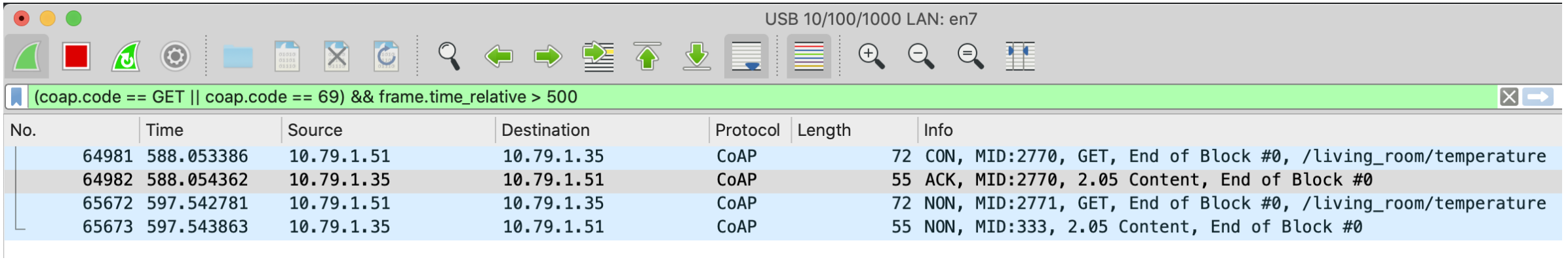
- Insert the filter: coap.code == POST || coap.code == 68



We obtain all packets containing CoAP POST Requests or 2.04 Responses

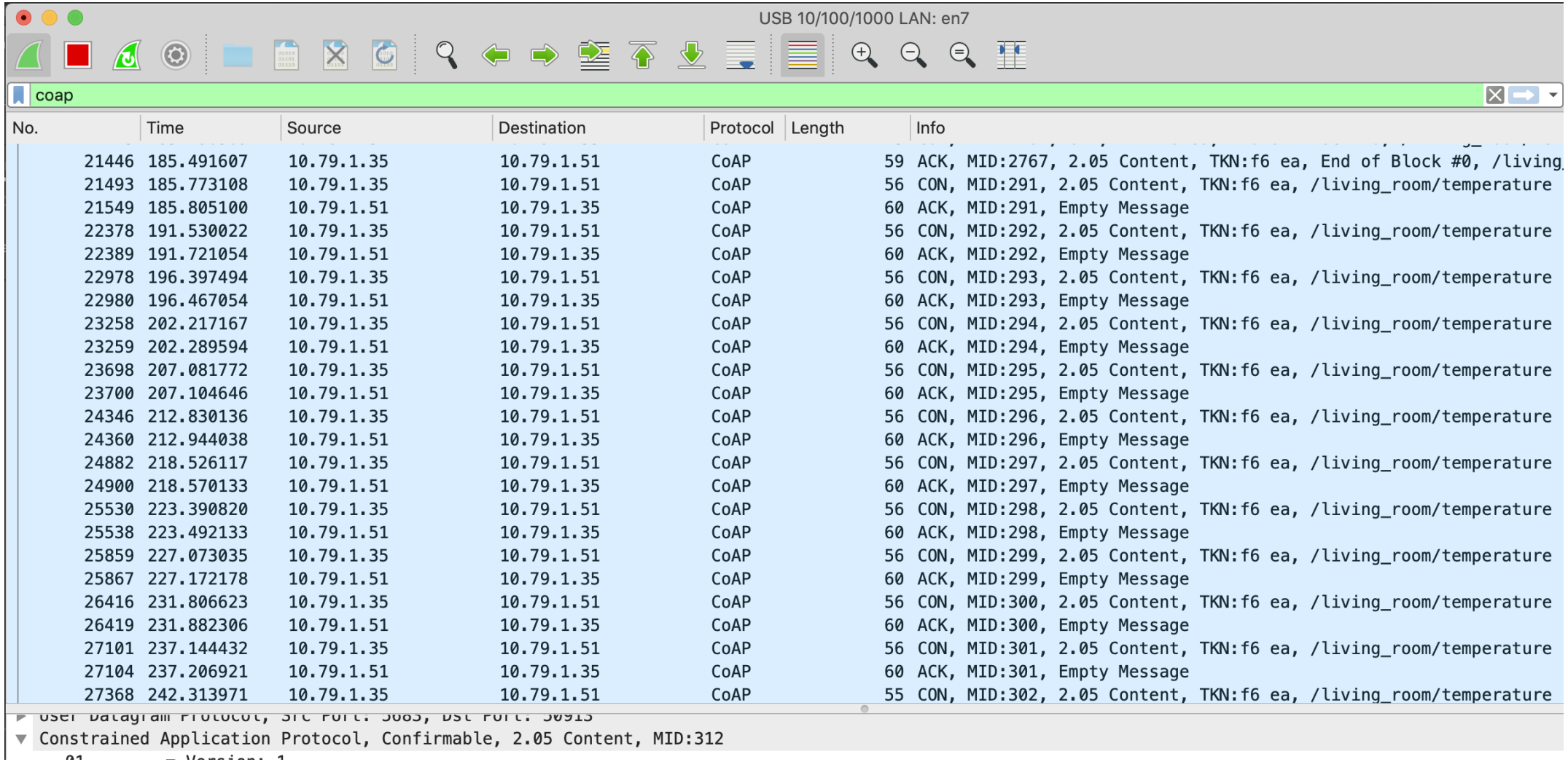
ANTLAB

CON vs. NON



ANTLAB

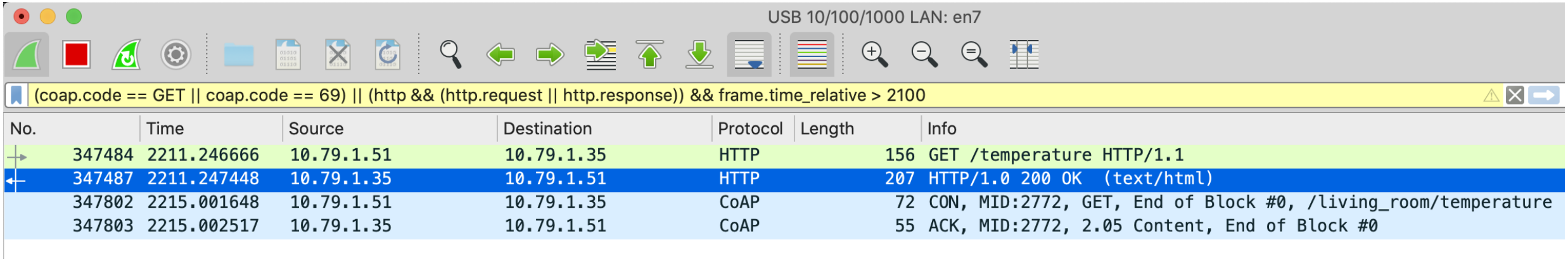
Observe



One request with many responses -> Same token, different MIDs

ANTLAB

HTTP vs. CoAP



No.	Time	Source	Destination	Protocol	Length	Info
347484	2211.246666	10.79.1.51	10.79.1.35	HTTP	156	GET /temperature HTTP/1.1
347487	2211.247448	10.79.1.35	10.79.1.51	HTTP	207	HTTP/1.0 200 OK (text/html)
347802	2215.001648	10.79.1.51	10.79.1.35	CoAP	72	CON, MID:2772, GET, End of Block #0, /living_room/temperature
347803	2215.002517	10.79.1.35	10.79.1.51	CoAP	55	ACK, MID:2772, 2.05 Content, End of Block #0

Hands on!

Open the *exercises.pcap* file in Wireshark

Find all the CoAP GET requests directed to non-existing resources

Code	Name	Reference
1	GET	[RFCXXXX]
2	POST	[RFCXXXX]
3	PUT	[RFCXXXX]
4	DELETE	[RFCXXXX]

CoAP Method Codes

Code	Description	Reference
65	2.01 Created	[RFCXXXX]
66	2.02 Deleted	[RFCXXXX]
67	2.03 Valid	[RFCXXXX]
68	2.04 Changed	[RFCXXXX]
69	2.05 Content	[RFCXXXX]
128	4.00 Bad Request	[RFCXXXX]
129	4.01 Unauthorized	[RFCXXXX]
130	4.02 Bad Option	[RFCXXXX]
131	4.03 Forbidden	[RFCXXXX]
132	4.04 Not Found	[RFCXXXX]
133	4.05 Method Not Allowed	[RFCXXXX]
134	4.06 Not Acceptable	[RFCXXXX]
140	4.12 Precondition Failed	[RFCXXXX]
141	4.13 Request Entity Too Large	[RFCXXXX]
143	4.15 Unsupported Media Type	[RFCXXXX]
160	5.00 Internal Server Error	[RFCXXXX]
161	5.01 Not Implemented	[RFCXXXX]
162	5.02 Bad Gateway	[RFCXXXX]
163	5.03 Service Unavailable	[RFCXXXX]
164	5.04 Gateway Timeout	[RFCXXXX]
165	5.05 Proxying Not Supported	[RFCXXXX]

CoAP Response Codes

Hands on! - SOLUTION

Open the *exercises.pcap* file in Wireshark

Find all the CoAP GET requests directed to non-existing resources

SOLUTION

- Filter with:

  coap.code==1 or coap.code==132

- You see all GET requests or 4.04 responses
- Count only GETs having a corresponding 4.04 response associating TOKENS